

PRD: Cupcake - Kafka Flow Testing Platform

Target: Enterprise-level Kafka testing and verification, fully dockerized with Angular frontend and Go backend.

Updated Tech Stack

Component	Technology
----- -----	
Frontend	Angular + RxJS + SCSS
Backend API	Go + Gin
Kafka Client	kafka-go (Segment)
Database	MongoDB (via mongo-go-driver)
Containers	Docker + Docker Compose

Core Concepts

Flows (Test Suites)

- A Flow (aka test suite) is a named test scenario
- Consists of multiple Paths (input + expected output pairs)
- Each flow can be saved, run, and verified later

Path (Single Test Step)

- Input: Kafka topic + message to produce
- Output: One or more expected messages with assertions
- Results are stored per path, so flows can be replayed

Features

Produce Message (Input)

- Define topic, key (optional), headers (optional), value (JSON)
- Send via UI or part of test flow

Subscribe to Topics (Consumer)

- Auto-consume from configured topics
- Each flow registers topics it wants to listen to
- Backend continuously polls and buffers incoming messages

Assertions

Expected Kafka responses can contain assert placeholders:

```
{  
  
  "status": "equals(SUCCESS)",  
  
  "timestamp": "any()",  
  
  "id": "not_empty()"
```

Supported assertions:

- equals(value)
- any() matches any value
- not_empty()
- greater_than(x) / less_than(x)
- contains(substr)

Flow Management

- Save flows to MongoDB with name, tags, and description
- View/edit past flows
- Re-run saved flows and see / status per path

UI (Angular App)

Main Views:

1. Dashboard - List of saved flows, run history
2. Flow Builder - Create/edit flow with input/output paths
3. Flow Runner - Run test, show results in real-time
4. Result Viewer - Expand details, show input/output/assertions

Docker Support

Use Docker Compose to spin up:

- backend (Go service)
- frontend (Angular)
- mongo (DB)
- (Optional) local Kafka or use external broker

docker-compose.yml will expose:

- Angular app at localhost:4200
- Go API at localhost:8080

Flow Example

```
{  
  
  "flow_name": "Order Processing",  
  
  "paths": [  
  
    {
```

```
"input": {  
  
  "topic": "order.create",  
  
  "value": {  
  
    "order_id": "abc123",  
  
    "amount": 200  
  
  }  
  
},  
  
"expected_outputs": [  
  
  {  
  
    "topic": "order.status",  
  
    "value": {  
  
      "order_id": "abc123",  
  
      "status": "equals(confirmed)",  
  
      "timestamp": "any()"  
  
    }  
  
  }  
  
]  
  
}
```

MongoDB Schema Proposal

flows collection

```
{  
  
  "_id": ObjectId,  
  
  "name": "Order Processing",
```

```
"paths": [ ... ],

"created_at": ISODate,

"last_run": ISODate,

"results": [

  {

    "path_index": 0,

    "status": "passed",

    "actual_output": {...},

    "logs": [...]

  }

]

}
```

Suggested Folder Structure

Backend (go/)

go/

main.go

routes/

flow.go

services/

kafka_producer.go

kafka_consumer.go

assertion.go

models/

flow.go

utils/

json_matcher.go

Frontend (angular/)

angular/

src/

app/

components/

services/

pages/

dashboard/

flow-builder/

test-runner/

Timeline Suggestion (for MVP)

Day | Task

----|-----

- 1 | Setup backend: API, Kafka produce/consume
- 2 | Setup Angular project with flow builder UI
- 3 | Add Kafka consumer buffering and assertion engine
- 4 | Full end-to-end test run support
- 5 | MongoDB persistence + history viewer
- 6-7 | UI polish, Docker Compose, docs, QA